Q1)

Ali Emre Kaya

150210097

y_true = [3.0, 2.0, 4.0, 5.0]

y_pred = [2.5, 2.0, 3.5, 6.0]

- Mean Absolute Error = $\frac{\sum_{i=1}^{n} |\hat{y}_i - y_i|}{n}$

$\frac{0.5+0+0.5+1}{4} = \frac{|3-2.5| + |2.0-2.0| + |4-3.5| + |5-6|}{4}$

↳ $2/4 = \boxed{0.5}$

- Mean Square Error = $\frac{\sum_{i=1}^{n} (\hat{y}_i - y_i)^2}{n}$

$\frac{(0.5)^2+(0)^2+(0.5)^2+(1)^2}{4} = \frac{0.25+0+0.25+1}{4}$

$\boxed{0.375} = \frac{1.5}{4}$ ↵

- R^2 (coefficient of determination) = $1 - \frac{SS_{res}}{SS_{tot}}$

$\bar{y}$ (average of y_true) = $\frac{3+2+4+5}{4} = \frac{14}{4} = 3.5$

$SS_{res} = \sum_{i=1}^{n} (\bar{y} - y_i)^2$

$SS_{tot} = \sum_{i=1}^{n} (\bar{y}_i - y_i)^2$

$SS_{tot} = (3.5-3)^2 + (3.5-2)^2 + (3.5-4)^2 + (3.5-5)^2$

$= 0.25 + 2.25 + 0.25 + 2.25$

$= 5$

$SS_{res} = (2.5-3)^2 + (2-2)^2 + (4-3.5)^2 + (6-5)^2$

$= 0.25 + 0 + 0.25 + 1$

$= 1.5$

$R^2 = 1 - \frac{1.5}{5} = \boxed{\frac{7}{10}}$

Keskin Color

Q2)

Ai: Emre Kayar

150210092

| Actual \ Prediction | True | False |
|---|---|---|
| True | 18 | 4 |
| False | 6 | 22 |

TP = 18
FP = 6
TN = 22
FN = 4

- Accuracy = $\frac{TN + TP}{TP + TN + FP + FN} = \frac{40}{50} = \underline{0.8}$

- Precision = $\frac{TP}{TP + FP} = \frac{18}{18 + 6} = \underline{0.75}$

- Recall = $\frac{TP}{TP + FN} = \frac{18}{18 + 4} = \underline{0.82}$

- F1 = $2 * \frac{Precision * Recall}{Precision + Recall} = 2 \frac{0.75 * 0.82}{0.75 + 0.82} = \underline{0.783}$

- TPR = $\frac{TP}{TP + FN} = \frac{18}{18 + 4} = \underline{0.82}$

- FPR = $\frac{FP}{FP + TN} = \frac{6}{6 + 22} = \underline{0.21}$

Koshin Color

Q3) Ali Emre Kaya - 150210097

a) Because linear models are scale-sensitive, features with different ranges (age: 0-100, salary: $0-10^6$) can lead to model learning errors and bias. StandardScaler within the pipeline aligns features to the same scale and prevents data leakage

b) age salary

```
X = [ [25, 50000],
      [50, 125000],
      [22, 30000], ... ]
```

- In this example, if the model is run without scaling the data, the salary will appear to be very important. To fix that, I can use scaler in pipeline

```
pipe = Pipeline([
        ("scaler": StandartScaler()),
        ("model": LogisticRegression()) ])

pipe.fit(X_train, y_train)
```

c) Accuracy is misleading in unbalanced data because the model can achieve high accuracy by predicting the majority. For example:

| actual \ pred | T | F |
|---|---|---|
| T | 99 | 0 |
| F | 1 | 0 |

In such a case, even model is trained to return true no matter what, it still provides 99% accuracy, which is incorrect. Instead of accuracy, F1 score may be a more effective score in such that datasets.

KoshinColor